

Sampling from an Energy: The Algorithms

Jaemoo Choi

- **The energy-specified target** The training methods of Lecture notes 3 and 5, written for a target specified by an energy E or its gradient ∇E — the tilted target $p_1^* \propto p_1^{\text{base}} e^{-E}$ when a pretrained model is given, the bare target $\nu \propto e^{-E}$ when nothing is. *Continuous* (Section 2): weighted denoising score matching, the proximal cross-entropy, adjoint matching, the bridge matching sampler. *Discrete* (Section 3): weighted target concrete score matching, the proximal cross-entropy on a chain, the soft cross-entropy, the discrete adjoint Schrödinger bridge sampler, the discrete bridge matching sampler. Algorithms only: what is simulated, what is weighted or labelled, what is regressed; every statement used here is proved in the Lecture 3 and Lecture 5 notes.

Date: August 31, 2026

1 The Energy-Specified Target

Notations E : the energy; Z : the unknown normalizer of whichever target is in force; μ : the drawable law at $t = 0$; time runs from noise ($t = 0$) to the target ($t = 1$); F : a test function; sg: stop-gradient. Section 2 (continuous) uses the Lecture 3 letters: a pretrained score network $\nabla \log p_t^{\text{base}}$ with noising pair (f_t, σ_t) , generative drift $b_t := -f_t + \sigma_t^2 \nabla \log p_t^{\text{base}}$ and path measure p^{base} ; W_t a Brownian motion; controls are σ -scaled (u^θ the trainee, $\bar{u}$ a behavior control, $u^{(k)}$ the proximal iterate's); a_t the adjoint state; γ_k , $\alpha_k = \gamma_k / (1 + \gamma_k)$ the proximal parameters. Section 2.5 rebinds several letters for the interpolant: points are lower case (x_0 , x_1 , x_t), γ_t is the interpolant schedule, b_t and s_t its velocity and score, b^θ and s^θ the learned fields, g_t a free noise level, c_t the blend, p_t the law of x_t , η a damping weight. Section 3 (discrete) uses the Lecture 5 letters: $\mathcal{X} = [N]^D$ the state space, x^d the d -th coordinate of x and $x^{d \leftarrow n}$ the state x with that coordinate set to n ; reference rates $b_t(y, x)$ with path measure p^{base} and transitions $p_{t|s}^{\text{base}}(y | x)$; controlled rates $u_t(y, x)$; network rows Φ^θ , Φ , $\hat{\Phi}$, C^θ indexed by (d, n) ; the Bregman divergence $\mathcal{D}$; the mask $\mathbb{M}$ with reveal schedule κ_t ; Schrödinger potentials φ , $\hat{\varphi}$; p^* the target path measure. Letter collisions, one meaning per place: $b_t(x)$ is the pretrained drift in Sections 2.1–2.4 and the interpolant velocity in Section 2.5, while $b_t(y, x)$ is always a rate; γ_t is the interpolant schedule in Section 2.5 and the uniform reference's noise schedule in Sections 3.5–3.6, while γ_k carries the proximal step; μ is the model's prior, the interpolant's Gaussian end, or the uniform or all-mask law. **The parent notes write the tilt $e^{\beta r}$; here the reward is the negative energy, $\beta r = -E$, so every formula below is the parent formula with $\beta r(X_1)$ replaced by $-E(X_1)$ and $\beta \nabla r$ by $-\nabla E$.**

Definition 1.1 (The two targets). Given an energy E with unknown normalizer,

$$\text{with a pretrained model: } p_1^* \propto p_1^{\text{base}} e^{-E}, \quad \frac{dp^*}{dp^{\text{base}}}(X) = \frac{e^{-E(X_1)}}{Z}, \quad Z = \mathbb{E}_{p^{\text{base}}} [e^{-E(X_1)}]; \quad (1)$$

$$\text{with nothing: } \nu(x) = \frac{e^{-E(x)}}{Z}, \quad \text{wanted: } x \sim \nu. \quad (2)$$

The tilt (1) depends on X_1 only, so it leaves the bridges alone, $p_{\cdot|1}^* = p_{\cdot|1}^{\text{base}}$, and p_t^* is the mixture of noising kernels $\int p_{t|1}^{\text{base}}(\cdot | x_1) p_1^*(x_1) dx_1$.

Access to the energy Z is never needed: every algorithm touches E through a Z -free reading. The continuous methods use the *value* $E(X_1)$ as an importance weight (Sections 2.2–2.3) or the *gradient* ∇E as a regression label (Sections 2.4–2.5). A finite state space has no gradient, so the discrete methods are all value-based: importance weights (Sections 3.2–3.3), or pointwise *differences* and softmaxes of E (Sections 3.4–3.6).

The two mechanisms The model-based methods carry the Radon–Nikodym derivative of $p^\star$ over what is simulated: reweighted by the value $e^{-E(X_1)}$ (Sections 2.2–2.3, 3.2–3.3), or minimized through the gradient ∇E (Section 2.4). The model-free methods carry no weight at all: they read a Z -free label off the energy at a self-generated endpoint and make the target a fixed point of $\text{simulate} \rightarrow \text{label} \rightarrow \text{regress}$ (Sections 2.5, 3.4–3.6).

2 The Continuous Methods

2.1 The Pretrained Model and the Proposals

The three equations The model, the behavior process, and the trainee are

$$p^{\text{base}} : dX_t = b_t(X_t) dt + \sigma_t dW_t, \quad X_0 \sim \mu, \quad b_t := -f_t + \sigma_t^2 \nabla \log p_t^{\text{base}}, \quad (3)$$

$$p^{\bar{u}} : dX_t = (b_t + \sigma_t \bar{u}_t)(X_t) dt + \sigma_t dW_t, \quad p^{u^\theta} : dX_t = (-f_t + \sigma_t u_t^\theta)(X_t) dt + \sigma_t dW_t, \quad (4)$$

both started at μ : the behavior control rides on the generative drift ($\bar{u} = 0$ simulates the model itself), while the trainee replaces the score wholesale.

2.2 Cross Entropy: Weighted Denoising Score Matching

The problem and the weight The objective is the forward Kullback–Leibler divergence, $\min_\theta D_{\text{KL}}(p^\star \| p^{u^\theta})$, over the trainee of (4): the unknown stands first, so the expectation is under the target — which cannot be sampled. An importance weight moves it to the behavior process, $\mathbb{E}_{p^\star}[F(X)] = \mathbb{E}_{p^{\bar{u}}}[\frac{dp^\star}{dp^{\bar{u}}}(X) F(X)]$ for every test function F , and the weight is explicit along the simulation (Lecture 3 note, Section 3.1):

$$\log \frac{dp^\star}{dp^{\bar{u}}}(X) = -E(X_1) - \int_0^1 \bar{u}_t^\top dW_t - \frac{1}{2} \int_0^1 \|\bar{u}_t\|^2 dt - \log Z. \quad (5)$$

$\log Z$ is a constant and disappears in a self-normalized (softmax) weight; with $\bar{u} = 0$ — simulate the pretrained model itself — every path term drops and the weight is just $-E(X_1)$.

The loss Draw $X^1, \dots, X^N \sim p^{\bar{u}}$ and softmax the weights; the weighted endpoints stand for $p_1^\star$, and the regression target is the conditional score of the noising kernel:

$$w^i = \text{softmax}_i \left(\log \frac{dp^\star}{dp^{\bar{u}}}(X^i) \right), \quad \mathcal{L}(\theta) = \sum_{i=1}^N w^i \mathbb{E}_{t, X_t} \left[\|u_t^\theta(X_t) - \sigma_t \nabla \log p_{t|1}^{\text{base}}(X_t | X_1^i)\|^2 \right], \quad (6)$$

with $t \sim \text{Unif}[0, 1]$ and $X_t \sim p_{t|1}^{\text{base}}(\cdot | X_1^i)$ — the endpoint noised back to time t ; the trajectory is thrown away, only the pairs (X_1^i, w^i) are kept. Its minimizer is $u_t^\theta = \sigma_t \nabla \log p_t^\star$, the drift that generates the tilted target (Lecture 3 note, Section 3.2); for linear f_t the kernel is Gaussian and the target is the usual noise prediction — weighted denoising score matching, that is, weighted flow matching.

Algorithm 1 Cross-entropy training, continuous (Phillips et al., 2024)

- 1: Simulate N particles of $p^{\bar{u}}$ from $X_0^i \sim \mu$, accumulating $\log w^i = -\bar{u}_t(X_t^i)^\top \Delta W_t^i + \frac{1}{2} \|\bar{u}_t(X_t^i)\|^2 \Delta t$.
 - 2: At $t = 1$: $w^i = \text{softmax}_i(\log w^i - E(X_1^i))$ ($\log Z$ drops in the softmax); keep the pairs (X_1^i, w^i) .
 - 3: Draw $t \sim \text{Unif}[0, 1]$, $X_t \sim p_{t|1}^{\text{base}}(\cdot | X_1^i)$, and take a gradient step on (6).
 - 4: Optionally set $\bar{u} \leftarrow u^\theta$ and repeat. ($\bar{u} = 0$: an offline batch of the pretrained model, weights $e^{-E(X_1^i)}$.)
-

2.3 The Proximal Cross-Entropy

The step and its weight One round of [algorithm 1](#) asks the weights to bridge the proposal all the way to $p^\star$, and the softmax concentrates on a few particles. The Kullback–Leibler proximal step moves a little at a

time (Lecture 3 note, Section 3.3): with $p^{(k)}$ the current model ($p^{(0)} = p^{\text{base}}$, $u^{(0)} = 0$) and $\gamma_k > 0$,

$$p^{(k+1)} := \arg \min_p \left\{ D_{\text{KL}}(p \| p^*) + \frac{1}{\gamma_k} D_{\text{KL}}(p \| p^{(k)}) \right\} \implies dp^{(k+1)} \propto (dp^*)^{\alpha_k} (dp^{(k)})^{1-\alpha_k}, \quad \alpha_k = \frac{\gamma_k}{1+\gamma_k}, \quad (7)$$

$$X \sim p^{(k)} : \quad \log w = \alpha_k \left(-E(X_1) - \int_0^1 u_t^{(k)}(X_t)^\top dW_t - \frac{1}{2} \int_0^1 \|u_t^{(k)}(X_t)\|^2 dt \right) + \text{const} \quad (8)$$

— the weight (5) at $\bar{u} = u^{(k)}$, tempered by α_k . Each iterate stays an energy tilt of the model, so the fixed point is p^* ; the price is many outer rounds.

Algorithm 2 Proximal cross-entropy training (Guo et al., 2026b)

- 1: **outer loop** ($k = 0, 1, 2, \dots$): simulate $X^1, \dots, X^N \sim p^{(k)}$, accumulating $\log w^i$ by (8).
 - 2: Set $w^i = \text{softmax}_i(\log w^i)$; keep the pairs (X_1^i, w^i) — a batch for the inner target $p^{(k+1)}$.
 - 3: **inner loop**: with the data fixed, draw $t \sim \text{Unif}[0, 1]$, $X_t \sim p_{t|1}^{\text{base}}(\cdot | X_1^i)$, and take gradient steps on (6) until converged.
 - 4: Set $u^{(k+1)} \leftarrow u^\theta$ and return to step 1.
-

2.4 Adjoint Matching

The control problem The reverse divergence of the tilted target is a stochastic control problem with terminal cost E (Lecture 3 note, Section 4.1):

$$D_{\text{KL}}(p^u \| p^*) = \mathbb{E}_{p^u} \left[\int_0^1 \frac{1}{2} \|u_t(X_t)\|^2 dt + E(X_1) \right] + \log Z, \quad (9)$$

minimized at $p^u = p^*$, attained by $u_t^* = \sigma_t \nabla \log H_t$ with $H_t(x) = \mathbb{E}_{p^{\text{base}}}[e^{-E(X_1)} | X_t = x]$. It is minimized by gradient steps through the simulation, so the *gradient* ∇E enters — the richer signal per sample; the price is on-policy trajectories.

The lean adjoint and the loss Along a simulated trajectory with the noise frozen, the adjoint state is a backward ODE — one vector–Jacobian product per step, no noise —

$$\frac{da_s}{ds} = -(\nabla_x b_s(X_s))^\top a_s, \quad a_1 = \nabla E(X_1) \quad \left(a_s = (\partial X_1 / \partial X_s)^\top \nabla E(X_1) \right), \quad (10)$$

and the optimal control is its conditional mean, $u_t^*(x) = -\sigma_t \mathbb{E}_{p^*}[a_t | X_t = x]$ (Lecture 3 note, Section 4.2). Hence the least squares

$$\mathcal{L}_{\text{AM}}(\theta) = \mathbb{E}_{X \sim p^{\text{sg}}(u^\theta)} \left[\int_0^1 \|u_t^\theta(X_t) + \sigma_t a_t\|^2 dt \right], \quad (11)$$

whose exact minimizer at the fixed point $p^{\text{sg}}(u^\theta) = p^*$ is u^* : simulate, then regress. Off-policy trajectories are corrected by the weights (5).

Algorithm 3 Adjoint matching (Domingo-Enrich et al., 2025)

- 1: Simulate $dX_t = (b_t + \sigma_t \text{sg}(u_t^\theta))(X_t) dt + \sigma_t dW_t$ from $X_0 \sim \mu$ and cache the trajectory — no gradient crosses it.
 - 2: Set $a_1 = \nabla E(X_1)$ and integrate (10) backward along the cached path: one vector–Jacobian product through b per step.
 - 3: Take a gradient step on (11).
 - 4: Repeat; at the fixed point $u^\theta = u^*$ and the simulated law is p^* .
-

2.5 The Bridge Matching Sampler

The equation The target is now bare, $\nu \propto e^{-E}$ with ∇E evaluable and no pretrained model, and this section rebinds b_t , γ_t and μ (see the Notations). The sampler (Blessing et al., 2026b) is one stochastic differential equation carrying two learned fields,

$$dx_t = [b_t^\theta(x_t) + g_t^2 s_t^\theta(x_t)] dt + \sqrt{2} g_t dW_t, \quad x_0 \sim \mu, \quad (12)$$

with b^θ the learned velocity, s^θ the learned score, and $g_t \geq 0$ a noise level chosen freely.

Definition 2.1 (The interpolant). Fix an increasing schedule γ_t with $\gamma_0 = 0$ and $\gamma_1 = 1$, and set

$$x_t := (1 - \gamma_t) x_0 + \gamma_t x_1, \quad (x_0, x_1) \sim \mu \otimes \nu \quad (\text{the two ends drawn independently}), \quad (13)$$

with p_t the law of x_t , so that $p_0 = \mu$ and $p_1 = \nu$ by construction (Albergo et al., 2025).

The guarantee The law p_t solves the transport equation of the velocity $b_t(x) = \mathbb{E}_{\mu \otimes \nu}[\dot{\gamma}_t(x_1 - x_0) \mid x_t = x]$, and for *every* noise level $g_t \geq 0$ the equation (12) driven by the true fields b_t and $s_t = \nabla \log p_t$ has exactly the marginals p_t — in particular $x_1 \sim \nu$ (Lecture 3 note, Section 6.2). The task is therefore to learn the two fields; the noise level and the Gaussian prior are free knobs, not modelling commitments.

The two labels and the loss Both fields are conditional expectations of quantities the problem grants (Lecture 3 note, Section 6.3): the velocity of $\dot{\gamma}_t(x_1 - x_0)$, and the score of the force at the endpoint,

$$\nabla \log p_t(x) = -\mathbb{E}_{\mu \otimes \nu} \left[\frac{1}{\gamma_t} \nabla E(x_1) \mid x_t = x \right] = \mathbb{E}_{\mu \otimes \nu} \left[\frac{1}{1 - \gamma_t} \nabla \log \mu(x_0) \mid x_t = x \right], \quad (14)$$

whose two expressions are blended with the balanced weight $c_t = \gamma_t^2 / (\gamma_t^2 + (1 - \gamma_t)^2)$ — the endpoint label blows up at $t \rightarrow 0$, the prior label at $t \rightarrow 1$, and the blend moves only the variance of the label, never its mean. With $t \sim \text{Unif}[0, 1]$, $(x_0, x_1) \sim \mu \otimes \nu$, and $x_t = (1 - \gamma_t)x_0 + \gamma_t x_1$ a deterministic function of the draws,

$$\begin{aligned} \mathcal{L}(\theta) &= \mathbb{E}_{t, \mu \otimes \nu} \left[\|b_t^\theta(x_t) - \dot{\gamma}_t(x_1 - x_0)\|^2 \right] \\ &\quad + \mathbb{E}_{t, \mu \otimes \nu} \left[\left\| s_t^\theta(x_t) - \frac{-\gamma_t \nabla E(x_1) + (1 - \gamma_t) \nabla \log \mu(x_0)}{\gamma_t^2 + (1 - \gamma_t)^2} \right\|^2 \right]. \end{aligned} \quad (15)$$

One training example costs one force evaluation $\nabla E(x_1)$, and nothing in (15) is a functional of a trajectory: no gradient ever crosses a simulation.

The self-generated endpoint (15) needs $x_1 \sim \nu$ — the one thing the problem does not give. The sampler substitutes the model’s own output: x_1 is produced by integrating (12) with the current fields, while x_0 is drawn afresh from μ . The score label still carries the true force $-\nabla E$, read at that endpoint — the model is corrected exactly where it puts its mass — and if the terminal law is already ν the regression returns the true fields: the target is a fixed point of the loop (Lecture 3 note, Section 6.4). Damping is what makes the loop behave: each round adds

$$\eta \mathbb{E}_{t, x_0, x_1} \left[\|b_t^\theta(x_t) - b_t^{\theta_i}(x_t)\|^2 + \|s_t^\theta(x_t) - s_t^{\theta_i}(x_t)\|^2 \right] \quad (16)$$

with the previous round’s parameters θ_i frozen, and a replay buffer of stored endpoints lets one force evaluation train many gradient steps.

Algorithm 4 The bridge matching sampler (Blessing et al., 2026b)

- 1: **Simulate.** Draw $x_0 \sim \mu$ and integrate (12) with the current fields to $t = 1$; keep only the endpoint x_1 .
 - 2: **Label.** Evaluate the force once and store the pair $(x_1, \nabla E(x_1))$ in a replay buffer.
 - 3: **Interpolate.** Draw a fresh $x_0 \sim \mu$ and a time $t \sim \text{Unif}[0, 1]$, and set $x_t = (1 - \gamma_t)x_0 + \gamma_t x_1$.
 - 4: **Regress.** Take a gradient step on $\mathcal{L}(\theta)$ of (15), damped by (16), and return to step 1.
-

JANUS [Blessing et al. \(2026a\)](#) run [algorithm 4](#) on disordered materials: the continuous state is every atom’s displacement from its lattice site together with the cell’s log-volume, b^θ and s^θ are two heads of one equivariant graph network conditioned on temperature and chemical potential, and the label is one call to an interatomic potential per stored endpoint — energy, forces, virial. Which species sits at which site is a discrete variable, and its channel is exactly the soft cross-entropy of [section 3.4](#).

3 The Discrete Methods

3.1 The Chain, the Rows, and the Divergence

Definition 3.1 (The chain and its rates). A rate $u_t(y, x) \geq 0$ ($y \neq x$) is the intensity of a jump *from* x *to* y at time t :

$$\Pr(X_{t+h} = y \mid X_t = x) = \delta(y, x) + u_t(y, x) h + o(h), \quad (17)$$

δ being the Kronecker delta. Simulation is by τ -leaping: over a step Δt , each single-coordinate move $x \rightarrow x^{d \leftarrow n}$ is taken with probability $u_t(x^{d \leftarrow n}, x) \Delta t$.

Rows and the regression divergence Every trained discrete object is a per-move *row* indexed by (coordinate, value) scaling the reference’s rates, $u_t(x^{d \leftarrow n}, x) = b_t(x^{d \leftarrow n}, x) \Phi_t(x)_{d,n}$, and rates are positive, so the least squares is replaced by the scalar Bregman divergence

$$\mathcal{D}(a \parallel \hat{a}) := a \log \frac{a}{\hat{a}} - a + \hat{a} \geq 0, \quad \text{zero iff } \hat{a} = a, \quad (18)$$

applied to one move at a time with the *label first*: like a least squares, a $\mathcal{D}$ -regression is minimized at the conditional mean of its label (Lecture 5 note, Section 3).

3.2 Cross Entropy: Weighted Target Concrete Score Matching

The problem and the weight The trainee is the ratio row on a pretrained discrete diffusion model, and the objective is again the forward Kullback–Leibler divergence:

$$u_t^\theta(y, x) = \Phi_t^\theta(x)_y b_t(y, x), \quad X_0 \sim \mu; \quad \min_\theta D_{\text{KL}}(p^\star \parallel p^{u^\theta}). \quad (19)$$

The importance weight over a behavior chain $p^{\bar{u}}$ is the chain Girsanov weight (Lecture 5 note, Section 3.1):

$$\log \frac{dp^\star}{dp^{\bar{u}}}(X) = -E(X_1) - \sum_{\text{jumps } t} \log \frac{\bar{u}_t}{b_t}(X_t, X_{t-}) + \int_0^1 \sum_{y \neq X_t} (\bar{u}_t - b_t)(y, X_t) dt - \log Z, \quad (20)$$

and with $\bar{u} = b$ — simulate the pretrained model itself — every path term drops and the weight is just $-E(X_1)$.

The loss The regression label is a ratio of the reference’s denoising kernels — the energy enters only through the weights:

$$\mathcal{L}(\theta) = \sum_{i=1}^N w^i \mathbb{E}_{t, X_t} \sum_{d=1}^D \sum_{n \neq X_t^d} \mathcal{D} \left(\frac{p_{1|t}^{\text{base}}(X_1^i \mid X_t^{d \leftarrow n})}{p_{1|t}^{\text{base}}(X_1^i \mid X_t)} \parallel \Phi_t^\theta(X_t)_{d,n} \right), \quad (21)$$

with $w^i = \text{softmax}_i$ of (20), $t \sim \text{Unif}[0, 1]$ and $X_t \sim p_{t|1}^{\text{base}}(\cdot \mid X_1^i)$. Its minimizer is the target’s concrete score, $\Phi_t(x)_{d,n} = \varphi_t(x^{d \leftarrow n})/\varphi_t(x)$ with $\varphi_t(x) = \mathbb{E}_{p^{\text{base}}}[e^{-E(X_1)} \mid X_t = x]$, that is $u^\theta = u^\star$ (Lecture 5 note, Section 3.2); for a masked reference the kernel ratio degenerates and (21) is the weighted denoising cross-entropy on the masked coordinates.

Algorithm 5 Cross-entropy training, discrete (Zhang et al., 2025)

- 1: τ -leap N particles with the rates $\bar{u}_t$ from $X_0^i \sim \mu$, accumulating the log-weight of (20): subtract $\log \frac{\bar{u}_t}{b_t}(X_t^i, X_{t-}^i)$ at a jump, add $\sum_{y \neq X_t^i} (\bar{u}_t - b_t) \Delta t$ while waiting. ($\bar{u} = b$: nothing to accumulate.)
 - 2: At $t = 1$: $w^i = \text{softmax}_i(\log w^i - E(X_1^i))$; keep the pairs (X_1^i, w^i) .
 - 3: Draw $t \sim \text{Unif}[0, 1]$ and $X_t \sim p_{t|1}^{\text{base}}(\cdot | X_1^i)$ — the endpoint noised back to time t .
 - 4: Take a gradient step on (21); optionally set $\bar{u} \leftarrow u^\theta$ and return to step 1.
-

3.3 The Proximal Cross-Entropy on a Chain

The tempered weight The proximal step (7) is channel-free — it acts on path measures — and on a chain its weight reads, with $X \sim p^{(k)}$ (rates $u^{(k)}$, $u^{(0)} = b$),

$$\log w = \alpha_k \left(-E(X_1) - \log \frac{dp^{(k)}}{dp^{\text{base}}}(X) \right) + \text{const}, \quad (22)$$

the log-ratio accumulated jump by jump as in (20) (Lecture 5 note, Section 3.3).

Algorithm 6 Proximal cross-entropy training on a chain (Guo et al., 2026b)

- 1: **outer loop** ($k = 0, 1, 2, \dots$; $u^{(0)} = b$): τ -leap $X^1, \dots, X^N \sim p^{(k)}$, accumulating $\log \frac{dp^{(k)}}{dp^{\text{base}}}$ as in (20).
 - 2: Set $w^i = \text{softmax}_i \left(\alpha_k \left[-E(X_1^i) - \log \frac{dp^{(k)}}{dp^{\text{base}}}(X^i) \right] \right)$; keep the pairs (X_1^i, w^i) — a batch for the inner target $p^{(k+1)}$.
 - 3: **inner loop**: with the data fixed, draw $t \sim \text{Unif}[0, 1]$, $X_t \sim p_{t|1}^{\text{base}}(\cdot | X_1^i)$, and take gradient steps on (21) until converged.
 - 4: Set $u^{(k+1)} \leftarrow u^\theta$ and return to step 1.
-

3.4 The Soft Cross-Entropy

The masked reference When the target is bare, $\nu \propto e^{-E}$ on $\mathcal{X} = [N]^D$, no weight is needed at all: the reference is a masked chain (Blessing et al., 2026a) — every coordinate starts at the mask symbol $\mathbf{M}$ and is revealed with probability κ_t by time t — and the network’s denoising row $C_t^\theta(x)_{d,n} \approx \Pr(X_1^d = n | X_t = x)$ fixes the rate,

$$u_t^\theta(x^{d \leftarrow n}, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} C_t^\theta(x)_{d,n}. \quad (23)$$

The label and the loss The label at a masked coordinate is the target’s own single-coordinate conditional, computable exactly from N energy evaluations,

$$\frac{\nu(x_1^{d \leftarrow n})}{\sum_m \nu(x_1^{d \leftarrow m})} = \text{softmax}_n \left[-E(x_1^{d \leftarrow n}) \right] = \nu(X_1^d = n | x_1^j, j \neq d) \quad (24)$$

— one coordinate given the others, Z cancelling — and the loss is the cross-entropy of the row against it: with X_1 drawn from the current model, $t \sim \text{Unif}[0, 1]$, and X_t obtained by re-masking each coordinate of X_1 with probability $1 - \kappa_t$,

$$\mathcal{L}(\theta) = -\mathbb{E}_{t, X_1, X_t} \sum_{d: X_t^d = \mathbf{M}} \sum_{n=1}^N \text{softmax}_n \left[-E(X_1^{d \leftarrow n}) \right] \log C_t^\theta(X_t)_{d,n}. \quad (25)$$

Its minimizer is the conditional mean of the label, and if the model’s endpoint law is ν that mean is $\nu(X_1^d = n | X_t)$ — the target is a fixed point of generate $\rightarrow$ label $\rightarrow$ regress (Lecture 5 note, Section 3.4). The label is exact pointwise: this is the contrast with (21), whose weights live on whole sequences.

Algorithm 7 The soft cross-entropy (Blessing et al., 2026a)

- 1: *Generate*: τ -leap the current model from the all-mask state and keep the endpoints $\text{sg}[X_1]$ — no sample of ν is used anywhere.
 - 2: *Label*: one batched call to E gives the N substituted energies $E(X_1^{d \leftarrow n})$ at every coordinate; store them with X_1 .
 - 3: *Interpolate*: draw $t \sim \text{Unif}[0, 1]$ and re-mask each coordinate of a stored X_1 with probability $1 - \kappa_t$.
 - 4: *Regress*: take a gradient step on (25) at the masked coordinates and return to step 1.
-

3.5 The Discrete Adjoint Schrödinger Bridge Sampler

The uniform reference Here too the target is bare, $\nu \propto e^{-E}$, and there is no pretrained model. The sampler (Guo et al., 2026a) — the chain version of the adjoint Schrödinger bridge sampler (Liu et al., 2025) — starts every chain at $\mu = \text{Unif}(\mathcal{X})$ and takes as reference the uniform chain on $\mathcal{X} = \mathbb{Z}_N^D$: with d_H the Hamming distance and γ_t a noise schedule,

$$b_t(y, x) = \frac{\gamma_t}{N} \quad \text{if } d_H(y, x) = 1, \quad b_t(y, x) = 0 \quad \text{otherwise } (y \neq x). \quad (26)$$

Its kernel depends on the difference alone, $p_{1|t}^{\text{base}}(y | x) = q_t(y - x)$ — the group structure of $\mathbb{Z}_N^D$ that both labels below use.

The bridge and its two networks The sampler solves the discrete Schrödinger bridge — the KL-closest chain to the reference among those transporting μ to ν — whose optimal rate is an h -transform by a pair of potentials (Lecture 5 note, Section 4.2):

$$u_t^*(y, x) = \frac{\varphi_t(y)}{\varphi_t(x)} b_t(y, x), \quad \varphi_1 \hat{\varphi}_1 = \nu, \quad \varphi_0 \hat{\varphi}_0 = \mu. \quad (27)$$

Since $\varphi_1 = \nu / \hat{\varphi}_1$ and the energy is known, only $\hat{\varphi}_1$ is genuinely unknown, and the ratio the sampler needs is computable without Z :

$$\frac{\varphi_1(z)}{\varphi_1(x_1)} = e^{E(x_1) - E(z)} \frac{\hat{\varphi}_1(x_1)}{\hat{\varphi}_1(z)}. \quad (28)$$

Two networks carry the two potentials as rows,

$$\Phi_t(x)_{d,n} \approx \frac{\varphi_t(x^{d \leftarrow n})}{\varphi_t(x)}, \quad \hat{\Phi}(x)_{d,n} \approx \frac{\hat{\varphi}_1(x^{d \leftarrow n})}{\hat{\varphi}_1(x)}, \quad u_t(x^{d \leftarrow n}, x) = \frac{\gamma_t}{N} \Phi_t(x)_{d,n}. \quad (29)$$

The two labels Both potentials are conditional means under the current model’s law p^* (Lecture 5 note, Section 4.4). The controller’s label is the endpoint *shifted by the move* — the discrete adjoint: no gradient, no backward pass, only differences of the energy —

$$\frac{\varphi_t(y)}{\varphi_t(x)} = \mathbb{E}_{p_{1|t}^*(x_1|x)} \left[\frac{\varphi_1(x_1 + y - x)}{\varphi_1(x_1)} \right], \quad (30)$$

its φ_1 ratios read off E and the frozen $\hat{\Phi}$ through (28); the corrector’s label is a reference kernel ratio — bridge matching at $t = 1$ —

$$\frac{\hat{\varphi}_1(z)}{\hat{\varphi}_1(y)} = \mathbb{E}_{p_{1|1}^*(x|y)} \left[\frac{p_{1|t}^{\text{base}}(z | x)}{p_{1|t}^{\text{base}}(y | x)} \right]. \quad (31)$$

Each is fitted by a $\mathcal{D}$ -regression with the label first:

$$\min_{\hat{\Phi}} \mathbb{E}_t \mathbb{E}_{p_{t,1}^*} \sum_{d, n \neq x^d} \mathcal{D} \left(\frac{\varphi_1(x_1^{d \leftarrow x_1^d + n - x^d})}{\varphi_1(x_1)} \parallel \Phi_t(x)_{d,n} \right) \quad (\text{adjoint matching}), \quad (32)$$

$$\min_{\hat{\Phi}} \mathbb{E}_t \mathbb{E}_{p_{t,1}^*} \sum_{d, n \neq x_1^d} \mathcal{D} \left(\frac{p_{1|t}^{\text{base}}(x_1^{d \leftarrow n} | x)}{p_{1|t}^{\text{base}}(x_1 | x)} \parallel \hat{\Phi}(x)_{d,n} \right) \quad (\text{corrector matching}). \quad (33)$$

Algorithm 8 The discrete adjoint Schrödinger bridge sampler (Guo et al., 2026a)

- 1: Initialize $\widehat{\Phi} \equiv 1$, so that $\varphi_1 = \nu$ and the first stage is the pure tilt.
 - 2: τ -leap N trajectories with the frozen rates $\text{sg}[(\gamma_t/N)\Phi]$ and keep the endpoints x_1 .
 - 3: Draw $t \sim \text{Unif}[0, 1]$ and bridge: $x \sim p_{t|0,1}^{\text{base}}(\cdot | x_0, x_1)$.
 - 4: Fit the controller Φ by (32), with the φ_1 ratios read off E and $\text{sg}[\widehat{\Phi}]$.
 - 5: Fit the corrector $\widehat{\Phi}$ by (33) on the same pairs (x, x_1) ; return to step 2. At the fixed point the two networks are the bridge's potentials.
-

3.6 The Discrete Bridge Matching Sampler

What is learned The chain version of the bridge matching sampler (Blessing et al., 2026b) gives up the Schrödinger coupling of section 3.5 and, with it, the corrector: one network, scaling the reference's moves, $u_t(x^{d \leftarrow n}, x) = b_t(x^{d \leftarrow n}, x) \Phi_t(x)_{d,n}$, with the uniform reference (26). The target measure is the reference's bridges glued under the *independent* coupling,

$$p^\star = \sum_{x_0, x_1} \mu(x_0) \nu(x_1) p^{\text{base}, |x_0, x_1}, \quad p_0^\star = \mu, \quad p_1^\star = \nu \text{ by construction}, \quad (34)$$

and the fitted rate is its Markovian projection $u_t^\star(y, x) = \mathbb{E}_{p^\star}[(p_{1|t}^{\text{base}}(X_1 | y)/p_{1|t}^{\text{base}}(X_1 | x)) b_t(y, x) | X_t = x]$, which keeps every marginal of $p^\star$ — this is why the fitted chain ends at ν (Lecture 5 note, Section 5.1).

The label On $\mathcal{X} = \mathbb{Z}_N^D$ the projection is one conditional expectation over the simulated triple (X_0, X_t, X_1) (Lecture 5 note, Section 5.2):

$$u_t^\star(y, x) = b_t(y, x) \mathbb{E}_{p^\star} \left[e^{E(X_1) - E(X_1 + y - x)} \cdot \frac{p_{1|0}^{\text{base}}(X_1 | X_0)}{p_{1|0}^{\text{base}}(X_1 + y - x | X_0)} \middle| X_t = x \right] \quad (35)$$

— the energy difference of the moved endpoint times the reference read end to end, Z cancelling. If the reference has mixed by $t = 1$, the second factor is 1 and the label is the bare energy difference: one network, one loss, nothing alternating.

The loss Write $y = x^{d \leftarrow n}$ for the move being rated. With the draws

$$t \sim \text{Unif}[0, 1], \quad x_1 \sim \text{the current chain at } t = 1, \quad x_0 \sim \mu, \quad x \sim p_{t|0,1}^{\text{base}}(\cdot | x_0, x_1), \quad (36)$$

the label of (35) is regressed with the label first,

$$\min_{\Phi} \mathbb{E}_{t, x_0, x_1, x} \sum_{d, n \neq x^d} \mathcal{D} \left(e^{E(x_1) - E(x_1 + y - x)} \frac{p_{1|0}^{\text{base}}(x_1 | x_0)}{p_{1|0}^{\text{base}}(x_1 + y - x | x_0)} \parallel \Phi_t(x)_{d,n} \right). \quad (37)$$

The fresh x_0 is what makes the coupling $\mu \otimes \nu$; if the current chain already ends at ν , the regression returns the Markovian projection itself — the target is a fixed point (Lecture 5 note, Section 5.3).

Algorithm 9 The discrete bridge matching sampler (Blessing et al., 2026b)

- 1: τ -leap N trajectories with the frozen rates $\text{sg}[b_t \Phi]$ and keep the endpoints x_1 .
 - 2: One batched call gives $E(x_1)$ and the $D(N-1)$ energies of its single-site substitutions — the same budget as the soft label of (25).
 - 3: Draw a fresh $x_0 \sim \mu$ and a time t , then bridge: $x \sim p_{t|0,1}^{\text{base}}(\cdot | x_0, x_1)$. The fresh x_0 is what makes the coupling $\mu \otimes \nu$: the model's own start is discarded.
 - 4: Take a gradient step on (37), moving Φ only part of the way to the new fit, and return to step 1.
-

Summary

	learns	the energy enters as	needs
<i>continuous (§2)</i>			
Weighted DSM (§2.2)	u^θ	the weight $e^{-E(X_1)}$	a pretrained model
Proximal cross-entropy (§2.3)	u^θ	the tempered weight $e^{-\alpha_k E(X_1)} \dots$	a pretrained model
Adjoint matching (§2.4)	u^θ	the gradient $\nabla E(X_1)$	a pretrained model
Bridge matching (§2.5)	b^θ and s^θ	the force $-\nabla E(x_1)$	nothing
<i>discrete (§3)</i>			
Weighted TCSM (§3.2)	the row Φ^θ	the weight $e^{-E(X_1)}$	a pretrained model
Proximal cross-entropy (§3.3)	the row Φ^θ	the tempered weight $e^{-\alpha_k E(X_1)} \dots$	a pretrained model
Soft cross-entropy (§3.4)	the row C^θ	the label $\text{softmax}_n[-E(x_1^{d \leftarrow n})]$	a mask
Discrete ASBS (§3.5)	Φ and $\hat{\Phi}$	the difference $e^{E(x_1)-E(z)}$	a group
Discrete bridge matching (§3.6)	Φ alone	the difference $e^{E(x_1)-E(x_1+y-x)}$	a group

Table 1 Nine training algorithms for a target specified by an energy: the tilted $p_1^* \propto p_1^{\text{base}} e^{-E}$ (rows with a pretrained model) or the bare $\nu \propto e^{-E}$ (the rest). All the model-free rows run one loop — simulate the current model, label from the energy at the endpoint, re-noise, regress — with the target a fixed point of the loop.

One target, $\propto e^{-E}$, and four Z -free entries of the energy:
a weight $e^{-E(X_1)}$ (§§2.2–2.3, 3.2–3.3), a softmax label (§3.4), the gradient ∇E (§§2.4–2.5), a difference (§§3.5–3.6).

References

- Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *Journal of Machine Learning Research*, 2025. arXiv:2303.08797.
- Denis Blessing, Mouyang Cheng, Maximilian Schebek, Jutta Rogal, Mingda Li, Carles Domingo-Enrich, and Yuanqi Du. JANUS: A multi-modal foundation neural sampler for disordered materials. *arXiv preprint arXiv:2608.19116*, 2026a.
- Denis Blessing, Lorenz Richter, Julius Berner, Egor Malitskiy, and Gerhard Neumann. Bridge matching sampler: Scalable sampling via generalized fixed-point diffusion matching. In *International Conference on Machine Learning (ICML)*, 2026b. BMS; arXiv:2603.00530.
- Carles Domingo-Enrich, Michal Drozdal, Brian Karrer, and Ricky T. Q. Chen. Adjoint matching: Fine-tuning flow and diffusion generative models with memoryless stochastic optimal control. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2409.08861.
- Guo, Zhu, Du, Nam, Chen, Gómez-Bombarelli, Liu, Tao, and Choi. Discrete adjoint Schrödinger bridge sampler. In *International Conference on Machine Learning (ICML)*, 2026a. DASBS; arXiv:2602.08243.
- Wei Guo, Jaemoo Choi, Yuchen Zhu, Molei Tao, and Yongxin Chen. Proximal diffusion neural sampler. In *International Conference on Learning Representations (ICLR)*, 2026b. arXiv:2510.03824.
- Guan-Hong Liu, Jaemoo Choi, Yongxin Chen, Benjamin Kurt Miller, and Ricky T. Q. Chen. Adjoint Schrödinger bridge sampler. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2506.22565.
- Angus Phillips, Hai-Dang Dau, Michael John Hutchinson, Valentin De Bortoli, George Deligiannidis, and Arnaud Doucet. Particle denoising diffusion sampler. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.06320.
- Ruixiang Zhang, Shuangfei Zhai, Yizhe Zhang, James Thornton, Zijing Ou, Joshua Susskind, and Navdeep Jaitly. Target concrete score matching: A holistic framework for discrete diffusion. In *International Conference on Machine Learning (ICML)*, 2025. TCSM; arXiv:2504.16431.